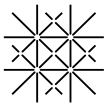

University of Basel

Interactive VR Museum for Image, Video and 3D Model Collections

Bachelor Thesis

Natural Science Faculty of the University of Basel
Department of Mathematics and Computer Science
Databases and Information Systems Group
https://dbis.dmi.unibas.ch/

Examiner: Prof. Dr. Heiko Schuldt
Supervisor: Rahel Arnold and Raphael Waltenspuel

Jannick Seper
jannick.seper@gmail.com
22-052-096

04.09.26

Abstract

Acknowledgments

Table of Contents

1 Introduction

Digital collections are usually shown on websites or as lists. These formats work well for direct searches, but do not fully provide the spatial and exploratory feel of an exhibition. This thesis studies how found media can instead be presented and explored in a virtual museum that users can move through freely.

1.1 Motivation

A digital collection can be accessed from any place and at any time. Standard search interfaces focus on a clear overview and direct access. On a screen, however, the media are shown next to each other in a two-dimensional format. This means that no real sense of space or connection between them is created.
Virtual Reality (VR)[1] offers a different form of access. Search results become exhibits. A person can move between them and view them from different angles. When a person finds an interesting exhibit, they can continue the search from it. In this way, searching becomes a repeatable path through the collection.
VIRTUE already shows images, videos and 3D artifacts in a VR museum [8]. A later extension automatically generates image galleries [20]. Since then, the vitrivr search backend has also been redesigned [7]. This thesis builds on the new vitrivr-engine and focuses on the VR interface rather than on the development of the search engine itself.

[1] *Virtual Reality* (VR) is a fully immersive environment that is experienced through a headset and can be explored interactively.

1.2 Initial Situation and Problem Statement

VIRTUE's automatic gallery focuses on image collections and uses Cineast and Cottontail DB, which are parts of the earlier vitrivr stack [20]. vitrivr-VR provides a fully immersive search, but it also uses these older parts [24]. MediaMix already uses the modular vitrivr-engine with PostgreSQL and pgvector, but it runs in Mixed Reality [1, 7]. None of these works shows a VR exhibition in which images, videos and 3D models are brought together from search results. In particular, it is still unclear how text-based and similarity searches can be used to repeatedly create and replace spatial exhibitions.
Images, videos and 3D models need different forms of presentation. The application must handle these differences while arranging all three media types in one exhibition.
The tested VR devices also differ in performance. The application either runs directly on the headset or is streamed from a PC. Input is provided through controllers or hand tracking. Despite these differences, the process in the museum should stay the same.
This situation leads to the following research question:

> *How can a multimodal collection be shown as a dynamic VR exhibition using the vitrivr-engine and explored through text and similarity searches?*

To answer this question, the VR application connects to the existing search backend. It loads results and shows each medium in a suitable form. Another search can then be started from an exhibited object. These steps should work in the same way on all tested devices.

1.3 Project Idea

The project idea is a virtual museum room whose content changes with every search. For this purpose, the backend provides an ordered list of images, videos and 3D models. The application counts the free places, loads the suitable selection and distributes it in the room. Images and videos hang in suitable frames on the walls and keep their aspect ratio. Videos start at the point that matches the search query. The 3D models stand in their own display places and can be viewed from several sides. When a new search starts, the room stays the same, but the exhibits change.
For each exhibit, the application also stores the vector through which it was found. When a person selects *Similar*, this vector starts a new similarity search. Its results replace the current exhibition. In this way, the collection can be explored step by step.
The museum room does not copy a real building. It serves as a spatial interface for digital collections that can be replaced.

1.4 Scope and Contributions

Scope This thesis studies a VR prototype with a prepared collection of images, videos and 3D models. It offers text and similarity searches and can be controlled with controllers or hand tracking. On the Meta Quest 3 and VIVE Focus 3, it runs directly as a standalone application. A Windows version can also be streamed to a supported headset. The work also includes the connection to vitrivr, installation instructions and a usability evaluation with a small group of test participants.
New search or embedding models are not part of this thesis. The same applies to a production-ready system, support for all VR devices and a copy of a real museum. The prototype uses existing components such as vitrivr-engine, Descriptor Server, CLIP, PostgreSQL and pgvector.

Contributions The contributions of this thesis are:

1. a concept that transfers search results into a spatial exhibition and connects text searches with similarity searches,
2. a VR prototype that shows images, found videos and 3D models together and can be controlled with controllers or hand tracking,
3. the connection of the prototype to the vitrivr-engine and the integration of the found media into the museum process and
4. an initial evaluation with a small group of test participants and instructions for reproducing the full setup.

The main contributions are the VR interface and the interaction between search, media and exhibition.

1.5 Thesis Outline

Chapter 2 presents related work on virtual exhibitions and immersive search systems. Chapter 3 explains multimedia retrieval, vector representations, CLIP, 3D model search and the vitrivr-engine. Chapter 4 describes the design of the VR museum. Chapter 5 presents the technical implementation. Chapter 6 describes the setup and results of the evaluation. Chapter 7 answers the research question, discusses the work and presents possible future extensions.

2
Related Work

This chapter explains how the thesis relates to previous research on virtual exhibitions and immersive search systems. It considers three areas that are important for the developed VR museum: virtual exhibitions, immersive multimedia retrieval and content-based exploration of virtual museums. Previous works cover individual parts of these areas. However, none of the reviewed works combines cross-media search, the dynamic creation of an exhibition and its continued exploration in an interactive VR museum.

2.1 Virtual Exhibitions

Virtual exhibitions solve a space problem. Digital collections can be much larger than the physical exhibition space that is available. In a virtual environment, the same objects can be used in different groups and can be accessed independently of their physical location. Two systems developed at the University of Basel are especially relevant to this thesis.

VIRTUE VIRTUE is a system for creating and viewing virtual exhibitions. An administrative interface allows users to create rooms and place images, videos and 3D artifacts. The exhibition is built with Unity and can then be visited in the VR frontend. Exhibitions, artifacts and their metadata are stored centrally in MongoDB. The system therefore follows a clear process. First, an exhibition is created manually. It is then viewed in VR. VIRTUE shows that a multimodal VR museum is technically possible, but it does not create an exhibition from a search query [8].

Automatic Gallery Generation Peterhans et al. [20] extend VIRTUE with the automatic creation of image galleries. The extended architecture consists of the Virtual Reality Exhibition Presenter (VREP), the web-based Virtual Reality Exhibition Manager User Interface (VREM-UI) and the Virtual Reality Exhibition Manager (VREM). VREM manages the exhibitions in MongoDB and provides them to the VR frontend through a REST interface. To create the galleries, VREM is connected to Cineast. The extracted image features are then stored in Cottontail DB. The components and their interfaces are shown in Figure 2.1 [20].

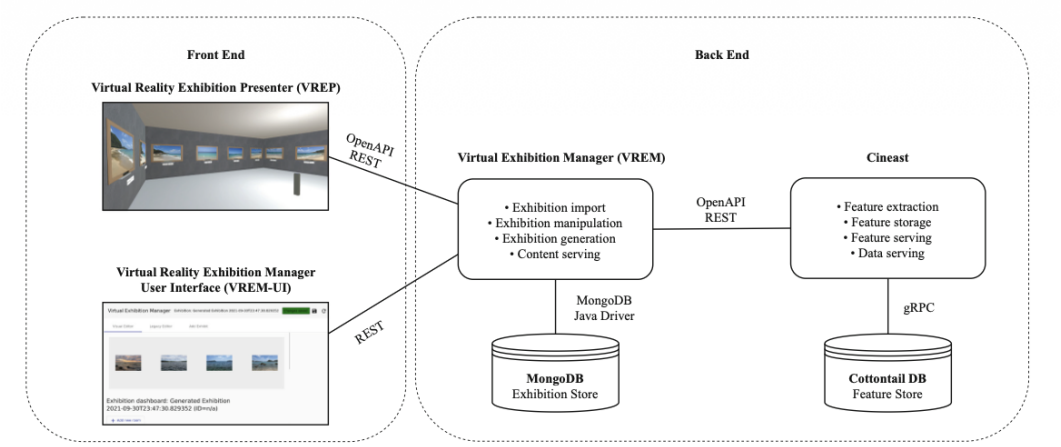

Figure 2.1: Architecture of VIRTUE with the extension for automatic image gallery creation [20, Fig. 1].

For the arrangement, the system trains a Self-Organizing Map on image features extracted earlier. Each node is assigned a representative image. When this image is selected, a sub-room with more images assigned to the same node is created [20].

Space-Adaptive Artwork Placement Kim et al. [15] follow an approach that focuses more on curating the exhibition. The system creates thematic groups of artworks and uses five adjustable content-similarity factors: color, material, description, artist and production date. An algorithm then determines the placement using four cost functions: intra-cluster distance, inter-cluster distance, intra-cluster intervisibility and occupancy. The process is summarized in Figure 2.2.

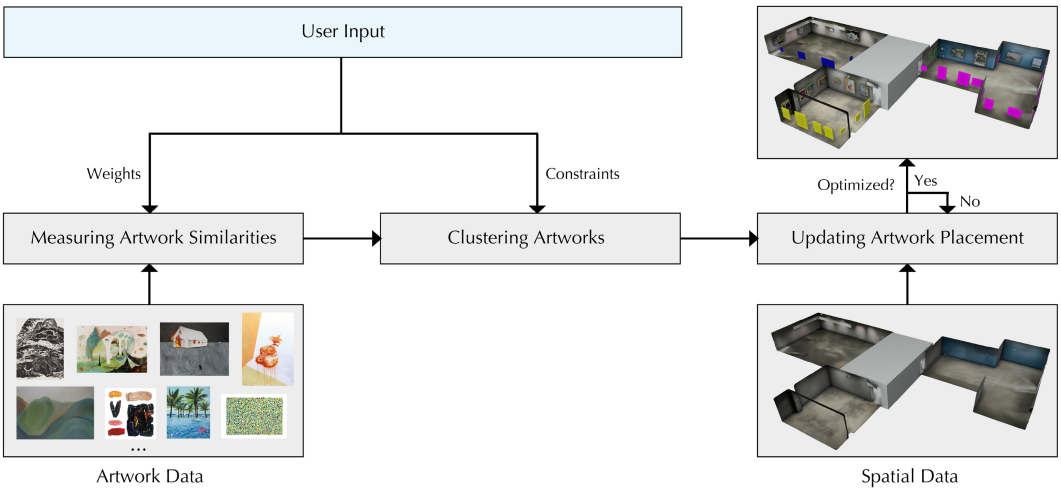

Figure 2.2: Similarity-based grouping followed by improved placement based on spatial rules [15, Fig. 1].

These works create exhibitions that are ordered by content or improved for the available space. The prototype developed here follows a different approach. Instead of curating an exhibition in advance, it builds a temporary exhibition from an already ranked result set from the vitrivr-engine. A new text or similarity query replaces this exhibition and continues the exploration at a new point in the collection.

2.2 Immersive Multimedia Retrieval

The second research area uses immersive environments mainly as user interfaces for multimedia retrieval, not as exhibition spaces. Both the creation of a query and the presentation and study of its results are moved into three-dimensional space.

vitrivr-VR vitrivr-VR is a fully immersive user interface for video search. The version described in 2022 uses Cottontail DB for storage and similarity search, Cineast for feature extraction and query processing and a VR frontend developed in Unity. The components communicate through the Cineast REST interface [24].
The system supports several types of queries: concept, text, Boolean, geographic, temporal and pose queries. Users enter text through an offline speech-to-text tool. The results appear on a cylindrical surface [24]. vitrivr-VR therefore shows how immersive forms of querying and result analysis can extend an existing retrieval system. This thesis follows the same basic idea. However, it only supports text and similarity queries, and it presents the results as an exhibition that users can walk through.

MediaMix MediaMix moves multimedia retrieval from a fully virtual environment into an MR environment[2]. The application uses the Apple Vision Pro and places search interfaces and results in the physical space. Its backend is the modular vitrivr-engine, which is connected to PostgreSQL and pgvector in the 2025 version. Figure 2.3 shows the separation into frontend, retrieval engine and database [1].

[2] *Mixed Reality* (MR) combines the visible real environment with spatially embedded digital content that users can interact with.

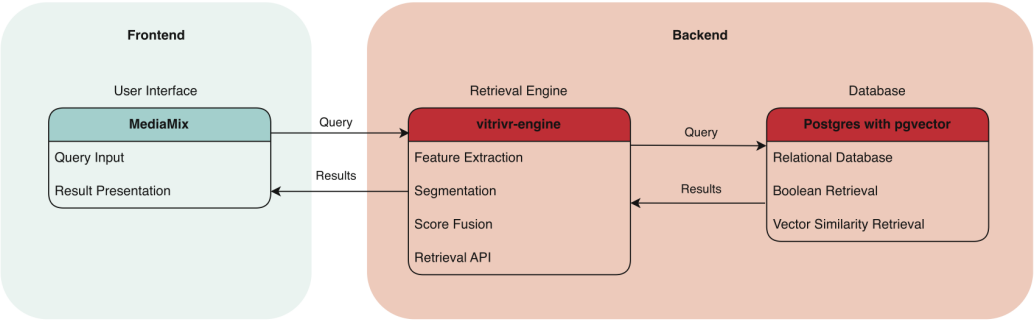

Figure 2.3: Architecture overview of the MediaMix stack [1, Fig. 1].

Text queries can be entered with a physical keyboard or Apple's speech-to-text service. Similarity queries use individual video frames and CLIP or DINO features[3]. CLIP is explained in Section 3.3. Each query creates its own result sphere, which stays in the room. This allows earlier and current searches to be studied side by side [1]. MediaMix and this thesis therefore both use a vitrivr backend and spatial retrieval interfaces. In the VR museum, images, videos and 3D models are presented together as exhibits and explored through a series of queries.

2.3 Content-Based Exploration in Virtual Museums

Systems in which a search query, a selected exhibit or observed behavior starts further exploration are especially close to the idea of this thesis.

Query-Based Exhibition Generation Hayashi et al. [13] already implement a direct process from a search query to a virtual exhibition. The application accepts keywords for fields in the Metropolitan Museum of Art's Open Access database, downloads the matching artwork images and automatically shows them in a 3D museum that visitors can move through freely. The basic idea of a search-based exhibition therefore already exists. However, this short system paper is limited to artwork images and metadata from a single external collection. It does not cover cross-media search or further exploration through an exhibited result.

Content-Based Navigation Koutsoudis et al. [16] connect a web-based virtual museum with a content-based search for 3D artifacts. The project contains 61 vessel models: six were digitized, while the remaining 55 were modeled by hand. Visitors can rotate an exhibited vessel, view information about it and use it for a query by example. A server then calculates the similarity of its shape to those of the other models. However, the system is a non-immersive application in a web browser, and its search is limited to a uniform collection of 3D vessels.

[3] DINO stands for *self-distillation with no labels* and is a self-supervised method for creating visual features.

Virtual Art Exhibition System The Virtual Art Exhibition System creates the next exhibition from a person's observed behavior. First, eight artworks are shown in a room. Invisible areas with a radius of two meters measure how long the person stays in front of each image. When the person leaves the room through an exit, the system identifies the work that was viewed for the longest time. It then searches for works related by period, artist and country and uses them to create a new exhibition. This change is shown in Figure 2.4 [5].

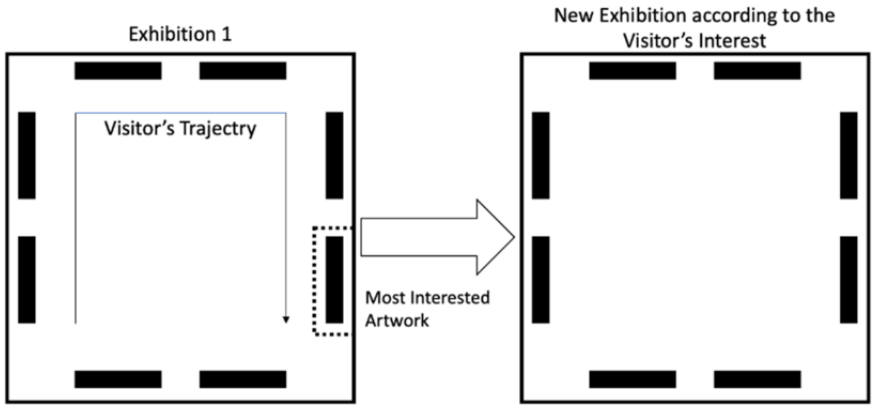

Figure 2.4: Creation of a new exhibition based on the artwork viewed for the longest time [5, Fig. 2].

Meinecke et al. [17] also focus on continued exploration. Their virtual museum links exhibited works to related images. Filters for artist, style and recognized object classes, as well as several interactive visualizations, offer different ways to access the collection. The work therefore combines a virtual museum, similarity calculation and exploratory search, but it remains an image-based online application.
These works are closer to the interaction model of the VR museum than a general method for 3D model search. They use a text query, a selected exhibit, observed interest or content filters to provide access to more works. This thesis lies where these approaches meet. Like VIRTUE, it presents different media types in VR. Like vitrivr-VR and MediaMix, it uses the retrieval backend. Like the systems for content-based navigation, it allows continued exploration. Its own contribution is to combine these features in one process. An exhibit directly starts a cross-media similarity search, and every result set appears as a new, fully immersive exhibition.

3
Background

This chapter describes the technical background of the VR museum. It first explains multimedia retrieval, semantic search and vector representations. It then presents CLIP and shows how the CLIP principle can be applied to 3D models. Finally, it describes the vitrivr-engine, which provides the search backend for this thesis.

3.1 Multimedia Retrieval

Multimedia retrieval means searching collections of images, videos, audio files or other media types. The search can use both existing metadata and the content of a medium. It can use features that can be calculated, such as color, texture, shape and motion [4]. vitrivr applies this principle to a modular retrieval stack with several query types [23].
In vitrivr-engine, a *descriptor* extracts a feature from a media item. This feature can be a single value, such as an average color value or a vector created by a neural network. A media item can have several such features [6, 7]. Vectors can generally capture more complex features of a medium.

3.2 Semantic Search and Vector Representations

A feature can be represented numerically as a feature vector. In the vector space model, a media object is shown as a point in a space with many dimensions [6]. A trained *embedding model* creates such vector representations [18, 22]. The similarity of two items is found from the distance between their vectors. In shared image-text embeddings, images and texts with related meanings should be close to each other [22, 26].
For a search, the query is also represented as a vector. The system compares it with the stored vectors and finds the nearest neighbors [6, 18]. Different distance functions can be used for this comparison. For example, Cottontail DB supports Manhattan, Euclidean and cosine distance, as well as other functions [6]. The descriptor and the backend decide which function is used. The nearest vectors form a ranked list, which is usually limited to the first k results. The ranking therefore describes similarity according to the feature that is used.

Figure 3.1 shows this principle in a simple example with three dimensions. The red query *Computer* is also represented as a vector. Of the stored terms, *CPU* is closest in meaning, so it appears first. *Web Development* is related too, but less closely, so it appears later. Real embeddings have many more than three dimensions. The figure therefore shows a simple projection, not the real feature space.

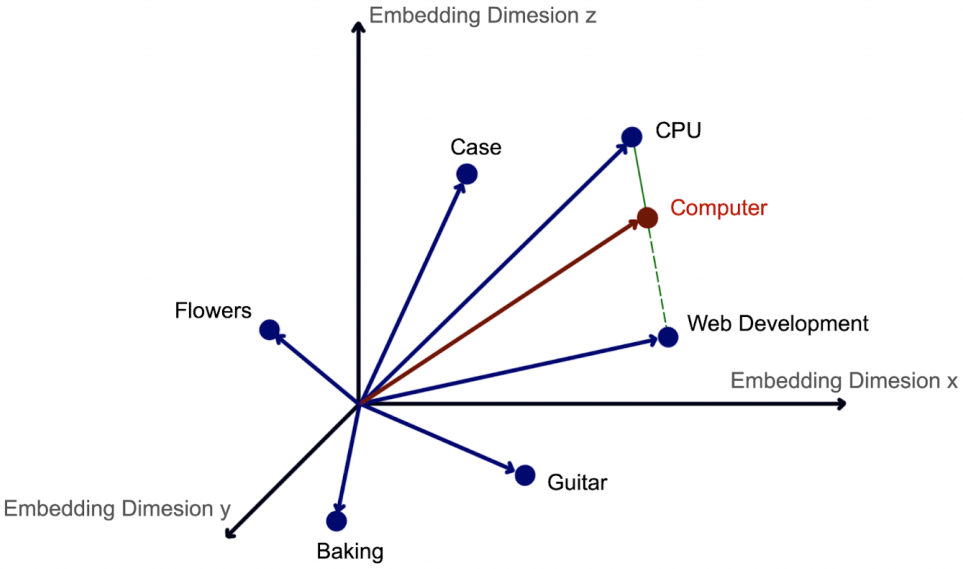

Figure 3.1: Simple view of a query and terms with similar meanings in a three-dimensional feature space. The green marks show the order of the most similar results.

The query vector can be created from different inputs. For a text search, the description is converted into a vector. For a similarity search, the stored vector of a media object can be used as the query. If no vector is available yet, the media object is embedded in the shared feature space with the same descriptor. The query vector is then compared with the stored vectors [4, 23].
This type of example search is often called *Query by Example* or *More Like This*. It does not need a new text description. Instead, it uses the representation of a selected result to find its nearest neighbors. In the VR museum, this principle is the basis of the *Similar* action. The vector stored with an exhibit becomes the query for the next search.

3.3 CLIP and Multimodal Content

For a direct comparison, text and visual media must be in the same feature space. CLIP stands for *Contrastive Language–Image Pre-training*. It uses an image encoder and a text encoder. The model is trained to match related images and texts within a batch and to separate pairs that do not belong together [22]. This creates comparable representations for images and texts.

OpenCLIP is one feature descriptor used by vitrivr-engine [7]. It provides an open-source implementation and a family of models that follow the CLIP principle [3]. The configuration in this thesis uses one field named `clip`, populated by a CLIP descriptor for all supported media types.

Figure 3.2 shows the general structure of a visual-text co-embedding in vitrivr. Video frames and text are processed through separate paths and then mapped into the same feature space. Matching videos and sentences should be close to each other, while videos that do not match should be farther away [24].

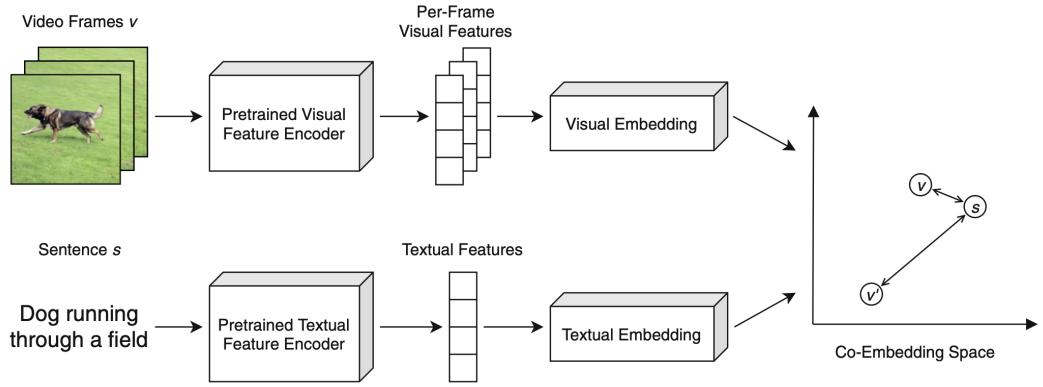

Figure 3.2: Processing of video frames and text in separate encoder paths and mapping into a shared co-embedding space [24, Fig. 2].

After training, new images and texts can be embedded independently. This allows a text query to be compared with the visual vectors in the collection. An image vector can also be used for a similarity search.

The original CLIP model processes images and texts. Individual frames can be used for videos. MediaMix uses this method for its similarity search [1].

3.4 Cross-Modal 3D Model Retrieval

A 3D model has features such as geometry, orientation, topology and textures. Models with different file formats, numbers of vertices or texture resolutions can still look almost the same when rendered. For retrieval, this complex representation must therefore be changed into compact features that can be compared [26].
Methods for describing 3D models can be divided roughly into geometry-based and view-based approaches. Geometry-based methods use features of the spatial structure. View-based methods instead use the visible appearance. For this purpose, the model is rendered from one or more camera positions. Because of perspective and hidden parts, a single view does not always show the full shape. However, it can be processed with existing image descriptors [26].
A well-known view-based method is *2D Planar View Mapping*, introduced by Chen et al. [2]. It represents a 3D model through projections from several viewpoints. Waltenspul et al. [26] build on this idea by processing such projections with image-text embedding models.

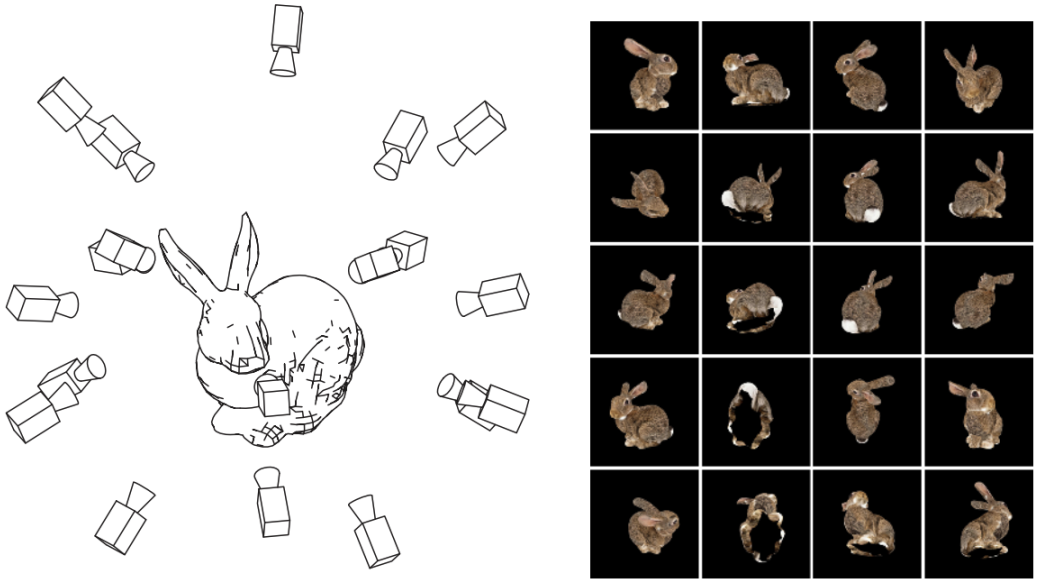

Figure 3.3: Creation of two-dimensional views of a 3D model [26, Fig. 2].

An image-text model converts each view into a vector, resulting in several vectors for one 3D model. Since some views may not show the object clearly, Waltenspul et al. [26] group similar vectors and combine the largest group into a single vector. This vector can then be compared with text-query vectors, enabling text-based retrieval of 3D models.

3.5 The vitrivr Retrieval Stack

vitrivr is an open-source stack for content-based multimedia retrieval. Today, the vitrivr-engine is its main backend. It connects media preparation and feature storage with later searches. The engine has a modular design and is not tied to a specific frontend or one media type [7]. In this thesis, the VR museum accesses the engine as a frontend.

Processing in the vitrivr-engine is split into configurable pipelines. Each pipeline is a sequence of operators with specific jobs, such as decoding a file, segmenting it, extracting a feature, searching vectors or loading additional data. The pipelines used to prepare the collection and run queries share the same data model and database connection [7].

During preparation, a media source is first recorded and decoded. Depending on the media type, more units can be created from it. For example, a video can be divided into time segments, while several views can be created for a 3D model. Descriptors are used to calculate the key features of each unit. The results and the relationships between them are then stored. By adjusting its configuration, the same engine can process images, videos and 3D models.

At the center of the data model are *Retrievables*. A Retrievable can stand for an entire media file or for a derived part, for example a temporal segment. Fields attached to a Retrievable hold values such as feature vectors or scalar attributes. Relations can link Retrievables, for example a video file and its segments [7].

The vitrivr-engine presented in 2024 used Cottontail DB [7]. MediaMix changed this setup in 2025 and uses PostgreSQL with the pgvector extension instead [1, 21]. This thesis follows the MediaMix setup. Images, videos and 3D models have separate indexing pipelines, but all three write their representations to the shared `clip` field. The vitrivr Python Descriptor Server produces the CLIP vectors. PostgreSQL stores them as vectors using pgvector and makes them available for similarity search [21, 25].

In the VR museum, the backend prepares the collection and processes both text-based and vector-based queries. It then returns a ranked list of results. The frontend uses this list to choose which items fit into the virtual room, present them as exhibits, and allow users to continue exploring through new searches.

4

Concept

Unlike a list or grid of results, the VR museum moves the search results into a space that visitors can walk through. It arranges the found images, videos and 3D models as an exhibition. Users can move freely, view exhibits from different angles and select exhibits for another search. The museum room stays the same, while the exhibition changes with every new query. This creates a continuous search process in which results are not only shown but can also be explored in space and used again.
The first part of this chapter describes how the full system is divided. It then explains how search results are put together as an exhibition based on the available space. The last section describes how users enter the museum and the repeatable exploration cycle.

4.1 Conceptual Architecture

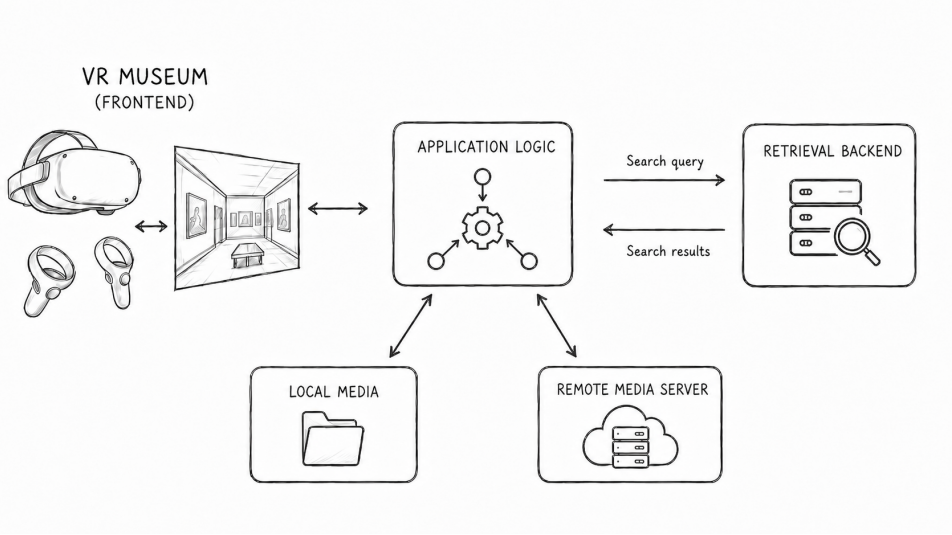

Figure 4.1: Conceptual architecture of the VR museum. The logic connects the frontend to the search function and provides the required media files [ChatGPT].

Figure 4.1 shows the conceptual architecture of the VR museum. The frontend includes the museum room and interaction through a VR device. It is designed for use on several VR platforms. The museum room, search process and basic interactions stay independent of the specific device. Users can enter text queries, view exhibits and select a medium that is already shown as the starting point of a similarity search.
The logic connects the museum to the other parts of the system. It turns the input into a search query, considers the media types and available space and prepares the results for the exhibition. The retrieval backend performs the semantic search and returns matching content. It does not decide where or how this content appears in the museum.
The collection is prepared before it is searched. Media are converted, checked and prepared for presentation. The backend then indexes the searchable representations. If the game engine is changed, the source media and search index stay available. Presentation files made for a specific frontend may have to be created again.
Media delivery is separate from the search process. Depending on the setup, the frontend either loads the file from the local system or gets it from an external server. This separation gives the frontend, application logic, search backend and media delivery clear roles. A component can be replaced as long as it uses the same interface and supports the same media formats.

4.2 Search-Based Exhibition Model

A query searches the prepared collection and returns a ranked list of results. Instead of showing this list directly, the museum uses the results to create a temporary exhibition in the virtual space.

4.2.1 Collection, Queries and Search Results

A *collection* includes all images, videos and 3D models that can be searched. The museum treats each file as a *media object*. For videos, a search result can point to a time segment, while a 3D model may be searched through images taken from different views. These results therefore need a link to the original media object that is shown in the museum.
Queries enter the system in two ways. Users may type a description or select a media object from an earlier result and use it as an example. In both cases, the backend produces ranked *search results*. A result identifies its media object and supplies the information required to build the corresponding exhibit.

4.2.2 Media-Specific Exhibits

An *exhibit* is the spatial presentation of a search result. It stays linked to its media object and can later be used as the starting point of a new search. Figure 4.2 shows the two basic forms for different media.

2D media. Images and videos appear as framed, two-dimensional exhibits. The content keeps its aspect ratio, while the frame adjusts to its dimensions. The frame has the same width on all four sides. This gives very narrow or wide media a consistent frame without changing the shape of their content.

Videos. In addition to the images, a video first shows a preview of the found segment. When a person comes closer, playback starts at the point that matches the query. This keeps the time reference of the search result in the exhibition.

3D media. 3D models appear inside a transparent glass sphere on a broken pillar. The sphere creates a clearly separated display space, so the model does not stand loosely on a surface. Within this space, it can be rotated and viewed from several sides without leaving its position in the exhibition. For a closer view, a model can be moved to a separate room. There, its size can be understood more easily without the other exhibits.

(a) Adjustable frame for two-dimensional media.

(b) Display place for a 3D model.

Figure 4.2: Media-specific presentation forms in the VR museum. The frame adjusts to the dimensions of an image or video. The pillar and glass sphere form a separate and scalable space for a 3D model.

4.2.3 Spatial Placement Areas

The positions for exhibits are already defined by the room. Wall areas are used for images and videos, while floor areas are used for 3D objects. A search result is placed in one of these existing areas and does not choose its own position. When a new search is started, only the exhibits change, while the room layout stays the same.

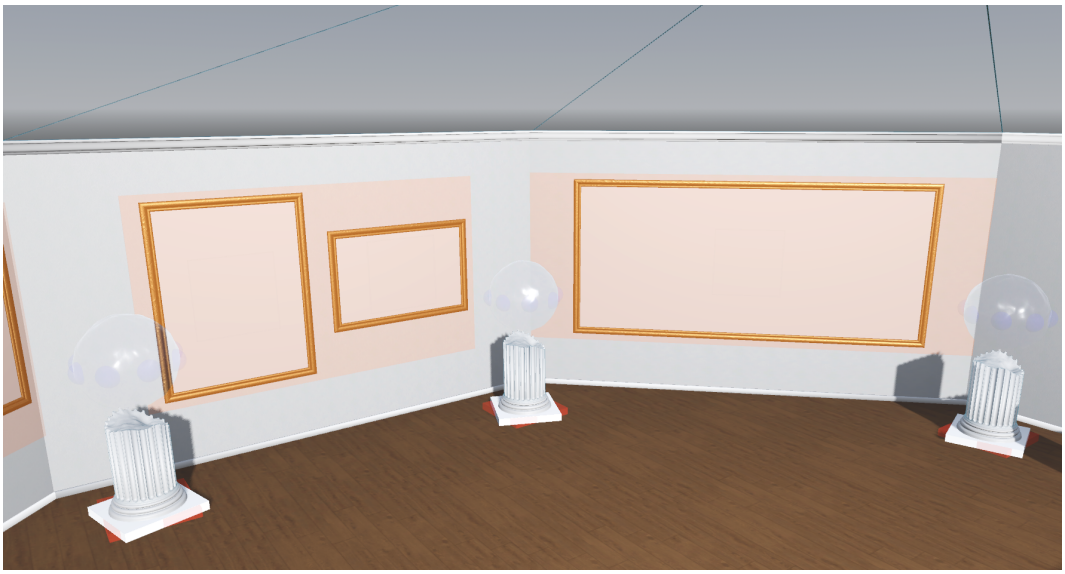

Figure 4.3: Defined placement areas in the museum room. The orange wall areas hold images and videos. The red floor areas define the position and size of the display places for 3D models, but not their rotation toward the center.

Figure 4.3 shows both types of areas. Each wall area has a maximum number of 2D exhibits. When a new exhibition starts, the application randomly chooses how many places to use. The wall is then divided into equal sections, with one frame placed in each section. This makes the walls look slightly different in each exhibition, while the limit keeps the exhibits clear and easy to view.

Each floor area contains one 3D exhibit. The area controls the dimensions of the whole display, including the pillar, glass sphere and model. Once this size is known, the model is scaled to remain inside the sphere. The same placement method can therefore handle small and large areas.

4.2.4 From Search Results to an Exhibition

An *exhibition* contains all exhibits created from one query. The collection itself does not change. Since space is limited, the room cannot reproduce the complete ranking and shows only the part that can be placed in its available areas.
Images and videos share the wall areas described above, while 3D models use the floor areas. Results are checked in ranking order. If a matching place is free and the media type is enabled, the result is included. Otherwise, it is skipped. In this way, no file is loaded without a place to display it, and the relative order of the included results is preserved.

4.2.5 Exhibition Replacement

Loading a new exhibition takes time. To avoid showing an empty room during this process, the old exhibition remains visible until the next result set and its files are ready. First, the current images and videos are removed, and the new 2D exhibits are placed step by step. The 3D exhibits follow. A failed query or response leaves the previous exhibition untouched. In contrast, a successful result with no media that can be shown clears the exhibition.

4.3 Exploration Model

The museum is designed for a sequence of searches. Its main concept is a cycle of query, exhibition and new query. This allows the collection to be explored step by step. Figure 4.4 shows this process.

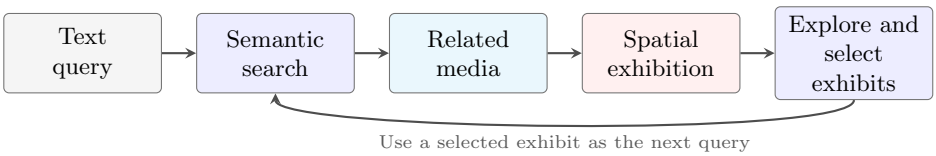

Figure 4.4: Continuous exploration cycle of the VR museum. A text query creates a spatial exhibition. A selected exhibit can then be used as the next query [ChatGPT].

4.3.1 Guided Entry and Settings

Before entering the museum, users first see a menu. There, they can check the connection to the backend. They can also select the desired media types and options for movement and presentation. An optional tutorial explains how to select and move objects, how to search, how to run a similarity search and how to view the size of a 3D model in a separate space. Actions can be started with VR controllers or, if the device supports it, with hand tracking. Both input methods use the same conceptual actions. The menu is separate from the museum room and can be opened again later without closing the current exhibition.

4.3.2 Text-Based Entry

The process starts with a text query. It defines a topic and creates the first exhibition, so it sets the starting point of the museum. The results can then be viewed in the exhibition.

4.3.3 Exploring the Exhibition

Users can move freely and view individual items more closely. Looking at an item in more detail does not affect the exhibition. A new exhibition is only created when an exhibit is selected as a new query.

4.3.4 Similarity-Based Continuation

Every exhibit can become the starting point of a similarity search. The *Similar* action searches for media objects that are similar to the selected exhibit. The search result creates a new exhibition.

A general text query can first open a broad topic. Selecting one exhibit makes the next search more specific. This process can be repeated any number of times and forms the main exploration cycle.

5 Implementation

Chapter 4 describes the concept of the VR museum. This chapter explains how it was implemented. It covers the choice of technology, the modular structure and the process from a search query to a new exhibition.

5.1 Technology Decisions

Godot and OpenXR OpenXR was chosen as a common cross-platform interface for several VR devices with different XR[4] runtimes [14]. The standard connects an XR application to the runtime that is used. Godot includes OpenXR as a core interface, and the implementation uses Godot 4.6.2 with .NET support [9, 10]. Godot XR Tools and the OpenXR Vendors Plugin are used as addons. Godot XR Tools provides user movement, object interaction and user-interface interaction. The OpenXR Vendors Plugin adds vendor-specific extensions and Android OpenXR loaders [11, 12]. The Meta Quest 3 and VIVE Focus 3 each use their own Android export profile. On Windows, the application uses the active OpenXR runtime and streams the image to the headset. The museum room and its interactions remain the same in every version.

Application Layers The C# source code is split across several .NET projects so that the search logic and media access are not tied to Godot. Godot handles presentation and input. The general application logic and the adapters for external services are in separate libraries. They do not depend on Godot scenes and are only put together with the current configuration at one central point.

Vitrivr and Media Delivery A new search engine was not part of this thesis, so the museum sends queries to the vitrivr-engine through its REST interface. Pipelines for images, videos and 3D models store their representations in the same CLIP field. The vitrivr Python Descriptor Server calculates these vectors [25]. PostgreSQL stores them as part of the Retrievables using pgvector. The media files are delivered separately through Nginx [19].

[4] *Extended Reality* (XR) is a general term for technologies such as VR and *Augmented Reality* (AR).

5.2 Implemented Architecture

Figure 5.1 divides the implementation into Godot integration, .NET libraries and external server services. Orange marks the factories, blue the Godot controllers, green the application and its use cases and turquoise the adapters and server components. The dashed arrows mark objects that the `MuseumApplicationFactory` creates and connects.

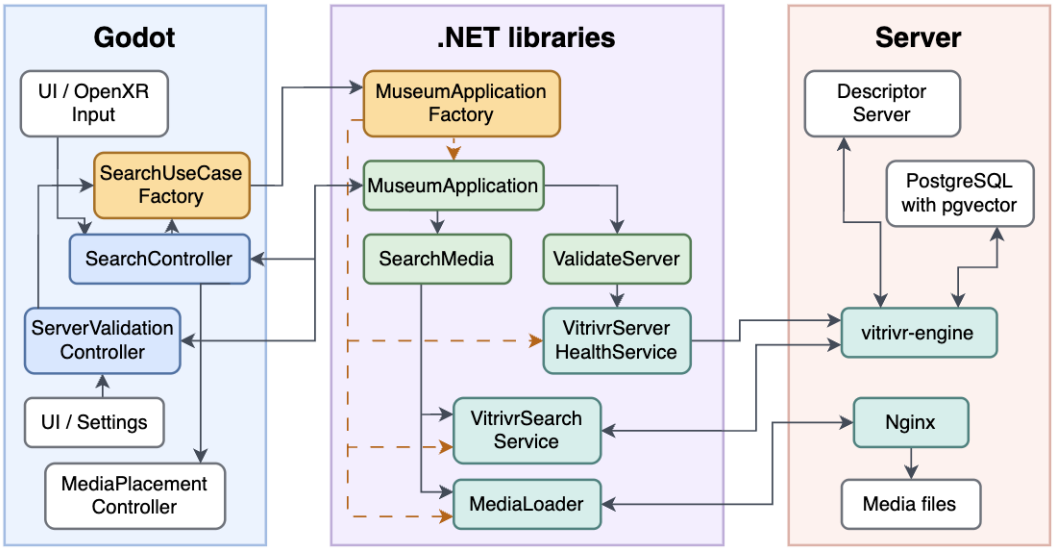

Figure 5.1: Component structure and composition of the Godot integration, .NET libraries and external services.

Godot manages input, settings and presentation. The .NET libraries handle the search process and access to external services. The servers perform the search, calculate descriptors, store data and deliver files.

The Godot controllers do not create HTTP requests or server adapters themselves. Instead, they receive a complete application and access it through one interface. This makes it clear in the architecture which parts need Godot and which can also work without the VR runtime.

5.3 Key Implementation Components

Godot Integration The `SearchController` accepts a query, prevents searches from running at the same time and finds the free places for 2D and 3D exhibits. After the search, it passes the loaded media to the strategies of the `MediaPlacementController`. `PlayerHandInput` turns controller and hand-tracking input into the same actions. The `ServerValidationController` checks from the settings menu whether the server can be reached. The *Original Size* action is handled by the `OriginalSizeController`. It moves the selected 3D model into a separate room and returns it afterward.

Application Layer Godot uses the `IMuseumApplication` interface to access the application logic. `MuseumApplication` sends search requests to `SearchMedia` and server checks to `ValidateServer`. `SearchMedia` needs a search engine and a service for loading media. `ValidateServer` only needs a service for checking the server. These interfaces separate the application logic from the vitrivr format, HTTP access and Godot.

Infrastructure and Server Integration The `VitrivrSearchService` acts as the boundary to the vitrivr-engine by translating queries and engine responses. When media are requested, the `MediaLoader` looks for a local file before downloading it through Nginx. Server checks remain separate from this search path and are handled by the `VitrivrServerHealthService`.

Object Composition Controllers do not create these services themselves. When a controller needs the application, it requests it from the `SearchUseCaseFactory`. The factory reads the current server and media paths from the Godot settings and passes them to the `MuseumApplicationFactory`. There, the `VitrivrSearchService`, `MediaLoader` and `VitrivrServerHealthService` are created. The factory then connects them to `SearchMedia`, `ValidateServer` and `MuseumApplication` and returns an `IMuseum Application`. After these objects are connected, the factory has finished its work. It does not start a separate runtime process. This setup keeps object creation inside the factories, so the other classes only need to know the components they directly use.

5.4 Runtime Flow

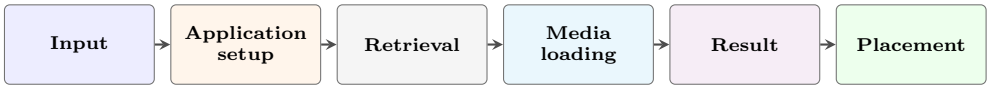

Figure 5.2: Simple pipeline of a successful search. The arrows show the order over time [ChatGPT].

After the server check succeeds, the first configured query runs once. For every text or similarity search, the controller asks the factory for an `IMuseumApplication`. This instance uses the current server and media paths. The controller then finds free display areas and calls `SearchMedia`. The use case creates the query from text or a vector that was stored earlier. The adapter then translates the query and response. The parser keeps the original ranking and removes duplicate paths. It links video segments to their full video files and replaces paths to 3D source files with paths to PCK packages[5]. After the application selects the allowed media types within the room capacity, the `MediaLoader` loads the missing files concurrently through Nginx.

Query or response errors leave the current exhibition in place. Individual exhibits with loading errors are skipped. In contrast, a valid response without media that can be shown clears the room. When placement succeeds, images and videos are added first, followed by 3D models. Only at the end does the `MediaLoader` release the files from the previous search. Because the available local storage is different on each device, this process prevents files from older searches from building up.

The 2D strategy chooses a random number for each wall area and adjusts the frame and collision area. A video first shows the found segment and starts at this point when a person comes closer. The 3D strategy loads and scales the PCK package. The CLIP vector stored with an exhibit is used by *Similar* as the next query.

[5] A PCK file is a Godot project package that contains imported resources and can be loaded at runtime.

5.5 Data Preparation and Deployment

Standalone headsets have less processing power and memory than the server used for the backend. The media are therefore optimized and standardized before indexing. The added pipeline backs up the source files and records its steps and errors. If the backup fails, it does not change any media files. Images are stored as JPG files with no more than 2048 pixels in each dimension. Videos are converted to OGV and limited to 2048 pixels along the longest dimension and 30 frames per second. 3D files are converted to GLB and checked. Animations, point clouds, invalid data and models with more than 1.5 million triangles are rejected directly. The presentation supports static, surface-based models. Animations and point clouds are therefore outside the planned format. Invalid or very complex files would also make reliable loading and good performance on standalone headsets more difficult.
Because direct GLB loading did not work reliably, Godot runs without a graphical interface and creates one PCK package for each model. vitrivr searches the GLB file, while the museum loads the imported scene from the PCK package.
The application has three export profiles. Separate Android applications are created for Quest and Focus. The Windows version is designed for PC streaming. Device-specific render profiles define resolution scaling, refresh rate and extra effects. The external `config.json` file stores the server address, tutorial option and first query. This makes it possible to use the same build with a different server or media collection.

6
Evaluation

7

Conclusion

Bibliography

[1] Rahel Arnold, Rahel Kempf, Raphael Waltenspül, and Heiko Schuldt. MediaMix: Multimedia retrieval in mixed reality. In Ichiro Ide, Ioannis Kompatsiaris, Changsheng Xu, Keiji Yanai, Wei-Ta Chu, Naoko Nitta, Michael Riegler, and Toshihiko Yamasaki, editors, *MultiMedia Modeling*, volume 15524, pages 310–317. Springer Nature Singapore, Singapore, 2025. ISBN 978-981-9620-73-9, 978-981-9620-74-6. doi: 10.1007/978-981-96-2074-6_37. URL https://link.springer.com/10.1007/978-981-96-2074-6_37.

[2] Ding-Yun Chen, Xiao-Pei Tian, Yu-Te Shen, and Ming Ouhyoung. On visual similarity based 3d model retrieval. *Computer Graphics Forum*, 22(3):223–232, September 2003. ISSN 0167-7055, 1467-8659. doi: 10.1111/1467-8659.00669. URL https://onlinelibrary.wiley.com/doi/10.1111/1467-8659.00669.

[3] Mehdi Cherti, Romain Beaumont, Ross Wightman, Mitchell Wortsman, Gabriel Ilharco, Cade Gordon, Christoph Schuhmann, Ludwig Schmidt, and Jenia Jitsev. Reproducible scaling laws for contrastive language-image learning. In *2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2818–2829, Vancouver, BC, Canada, June 2023. IEEE. ISBN 979-8-3503-0129-8. doi: 10.1109/CVPR52729.2023.00276. URL https://ieeexplore.ieee.org/document/10205297/.

[4] Myron Flickner, Harpreet Sawhney, Wayne Niblack, Jonathan Ashley, Qian Huang, Byron Dom, Monika Gorkani, Jim Hafner, Denis Lee, Dragutin Petkovic, David Steele, and Peter Yanker. Query by image and video content: The QBIC system. *Computer*, 28(9):23–32, September 1995. ISSN 0018-9162. doi: 10.1109/2.410146. URL http://ieeexplore.ieee.org/document/410146/.

[5] Tsukasa Fukuda and Naoki Ishibashi. Virtual art exhibition system: An implementation method for creating an experiential museum system in a virtual space. In Marina Tropmann-Frick, Hannu Jaakkola, Bernhard Thalheim, Yasushi Kiyoki, and Naofumi Yoshida, editors, *Frontiers in Artificial Intelligence and Applications*. IOS Press, January 2023. ISBN 978-1-64368-370-6, 978-1-64368-371-3. doi: 10.3233/FAIA220491. URL https://ebooks.iospress.nl/doi/10.3233/FAIA220491.

[6] Ralph Gasser, Luca Rossetto, Silvan Heller, and Heiko Schuldt. Cottontail db: An open source database system for multimedia retrieval and analysis. In *Proceedings of the 28th ACM International Conference on Multimedia*, pages 4465–4468, Seattle, WA, USA, October 2020. ACM. ISBN 978-1-4503-7988-5. doi: 10.1145/3394171.3414538. URL https://dl.acm.org/doi/10.1145/3394171.3414538.

[7] Ralph Gasser, Rahel Arnold, Fynn Faber, Heiko Schuldt, Raphael Waltenspül, and Luca Rossetto. A new retrieval engine for vitrivr. In Stevan Rudinac, Alan Hanjalic, Cynthia Liem, Marcel Worring, Bjorn Thor Jonsson, Bei Liu, and Yoko Yamakata, editors, *MultiMedia Modeling*, volume 14557, pages 324–331. Springer Nature Switzerland, Cham, 2024. ISBN 978-3-031-53301-3, 978-3-031-53302-0. doi: 10.1007/978-3-031-53302-0_28. URL https://link.springer.com/10.1007/978-3-031-53302-0_28.

[8] Ivan Giangreco, Loris Sauter, Mahnaz Amiri Parian, Ralph Gasser, Silvan Heller, Luca Rossetto, and Heiko Schuldt. VIRTUE: a virtual reality museum experience. In *Companion Proceedings of the 24th International Conference on Intelligent User Interfaces*, pages 119–120, Marina del Rey, California, March 2019. ACM. ISBN 978-1-4503-6673-1. doi: 10.1145/3308557.3308706. URL https://dl.acm.org/doi/10.1145/3308557.3308706.

[9] Godot Engine. Download godot 4.6.2 (stable), n.d. URL https://godotengine.org/download/archive/4.6.2-stable/. Accessed: August 4, 2026.

[10] Godot Engine documentation. Setting up XR, n.d. URL https://docs.godotengine.org/en/stable/tutorials/xr/setting_up_xr.html. Accessed: August 4, 2026.

[11] Godot Engine documentation. Introducing XR tools, n.d. URL https://docs.godotengine.org/en/stable/tutorials/xr/introducing_xr_tools.html. Accessed: August 4, 2026.

[12] Godot OpenXR Vendors plugin documentation. Welcome to the godot OpenXR vendors plugin documentation! – godot OpenXR vendors plugin documentation, n.d. URL https://godotvr.github.io/godot_openxr_vendors/. Accessed: August 4, 2026.

[13] Masaki Hayashi, Steven Bachelder, and Masayuki Nakajima. Automatic generation of art exhibitions from the internet resources. In *2020 IEEE 9th Global Conference on Consumer Electronics (GCCE)*, pages 642–643, Kobe, Japan, October 2020. IEEE. ISBN 978-1-7281-9802-6. doi: 10.1109/GCCE50665.2020.9291722. URL https://ieeexplore.ieee.org/document/9291722/.

[14] Khronos Group. OpenXR – high-performance access to AR and VR – collectively known as XR – platforms and devices, December 2016. URL https://www.khronos.org/openxr/. Accessed: August 4, 2026.

[15] Hayun Kim, Maryam Shakeri, Jae-Eun Shin, and Woontack Woo. Space-adaptive artwork placement based on content similarities for curating thematic spaces in a virtual museum. *Journal on Computing and Cultural Heritage*, 17(1):1–21, February 2024. ISSN 1556-4673, 1556-4711. doi: 10.1145/3631134. URL https://dl.acm.org/doi/10.1145/3631134.

[16] Anestis Koutsoudis, Christina Makarona, and George Pavlidis. Content-based navigation within virtual museums. *Journal of Advanced Computer Science & Technology*, 1(2):73–81, May 2012. ISSN 2227-4332. doi: 10.14419/jacst.v1i2.135. URL http://www.sciencepubco.com/index.php/JACST/article/view/135.

[17] Christofer Meinecke, Chris Hall, and Stefan Jänicke. Towards enhancing virtual museums by contextualizing art through interactive visualizations. *Journal on Computing and Cultural Heritage*, 15(4):1–26, December 2022. ISSN 1556-4673, 1556-4711. doi: 10.1145/3527619. URL https://dl.acm.org/doi/10.1145/3527619.

[18] Solmaz Seyed Monir, Irene Lau, Shubing Yang, and Dongfang Zhao. VectorSearch: Enhancing document retrieval with semantic embeddings and optimized search, September 2024. URL https://arxiv.org/abs/2409.17383.

[19] NGINX Documentation. Serve static content — NGINX documentation, n.d. URL https://docs.nginx.com/nginx/admin-guide/web-server/serving-static-content/. Accessed: August 4, 2026.

[20] Simon Peterhans, Loris Sauter, Florian Spiess, and Heiko Schuldt. Automatic generation of coherent image galleries in virtual reality. In Gianmaria Silvello, Óscar Corcho, Paolo Manghi, Giorgio Maria Di Nunzio, Koraljka Golub, Nicola Ferro, and Antonella Poggi, editors, *Linking Theory and Practice of Digital Libraries*, volume 13541, pages 282–288. Springer International Publishing, Cham, 2022. ISBN 978-3-031-16801-7, 978-3-031-16802-4. doi: 10.1007/978-3-031-16802-4_23. URL https://link.springer.com/10.1007/978-3-031-16802-4_23.

[21] pgvector contributors. pgvector, July 2026. URL https://github.com/pgvector/pgvector. Accessed: July 30, 2026; original date: April 20, 2021.

[22] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. Learning transferable visual models from natural language supervision. arXiv preprint arXiv:2103.00020, 2021. URL https://arxiv.org/abs/2103.00020.

[23] Luca Rossetto, Ivan Giangreco, Claudiu Tanase, and Heiko Schuldt. vitrivr: A flexible retrieval stack supporting multiple query modes for searching in multimedia collections. In *Proceedings of the 24th ACM International Conference on Multimedia*, pages 1183–1186, Amsterdam, The Netherlands, October 2016. ACM. ISBN 978-1-4503-3603-1. doi: 10.1145/2964284.2973797. URL https://dl.acm.org/doi/10.1145/2964284.2973797.

[24] Florian Spiess, Ralph Gasser, Silvan Heller, Mahnaz Parian-Scherb, Luca Rossetto, Loris Sauter, and Heiko Schuldt. Multi-modal video retrieval in virtual reality with vitrivr-VR. In Bjorn Thor Jonsson, Cathal Gurrin, Minh-Triet Tran, Duc-Tien Dang-Nguyen, Anita Min-Chun Hu, Binh Huynh Thi Thanh, and Benoit Huet, editors, *MultiMedia Modeling*, volume 13142, pages 499–504. Springer International Publishing, Cham, 2022. ISBN 978-3-030-98354-3, 978-3-030-98355-0. doi: 10.1007/978-3-030-98355-0_45. URL https://link.springer.com/10.1007/978-3-030-98355-0_45.

[25] vitrivr. vitrivr/vitrivr-python-descriptor-server, November 2025. URL https://github.com/vitrivr/vitrivr-python-descriptor-server. Accessed: August 4, 2026; original date: September 13, 2023.

[26] Raphael Waltenspul, Florian Spiess, and Heiko Schuldt. Cross-modal 3d model retrieval. In *2024 International Symposium on Multimedia (ISM)*, pages 176–180, Tokyo, Japan, December 2024. IEEE. ISBN 979-8-3315-1111-1. doi: 10.1109/ISM63611.2024.00039. URL https://ieeexplore.ieee.org/document/10935974/.

List of Figures

A

Use of Generative Artificial Intelligence

OpenAI Codex and ChatGPT with the GPT-5.5 and GPT-5.6 Sol models were used for different tasks in this report. These tasks included text corrections, translations and making difficult sentences easier to understand. The tools also provided extra help with technical questions. They helped compare statements from the cited sources, understand concepts and existing source code and review different ways to describe or implement a problem.
The generated answers were not used without review. They were compared with the sources, the source code and the author's own results. They were changed when needed. The author made all decisions about the implementation, the evaluation and the interpretation of the results.
The use of generative AI in the project itself, especially for source code, technical documentation, and project illustrations, is documented separately in the *Use of AI during development* section of the repository README.[6]
Generative AI was also used to create or design individual figures. ChatGPT created the conceptual architecture in Figure 4.1 based on the author's content and visual instructions. ChatGPT also helped arrange and present the boxes in Figures 4.4 and 5.2.

[6] Project repository and full AI declaration: README in the project repository.